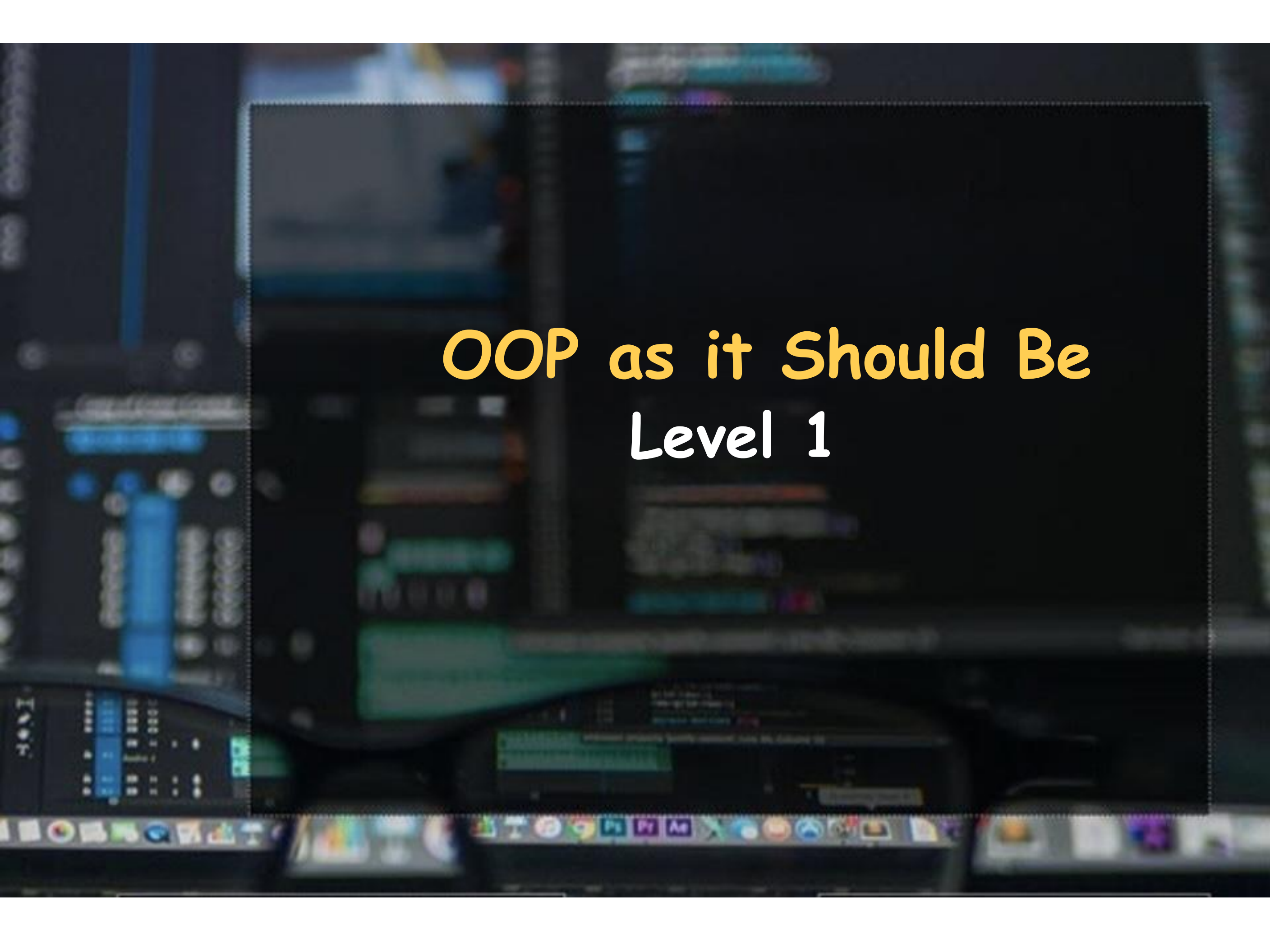
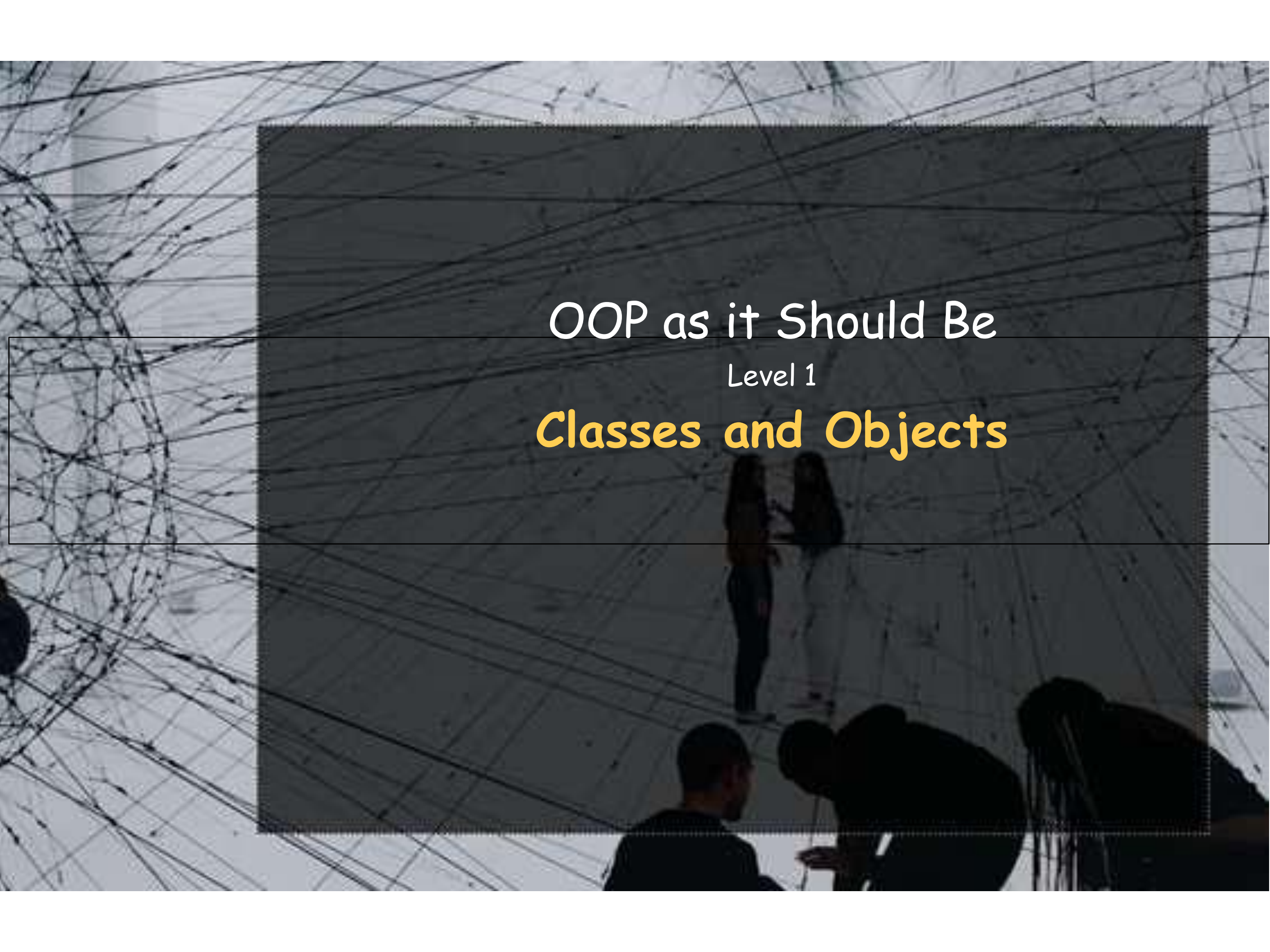




OOP as it Should Be Level 1

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a discussion or collaborative work, which adds a human element to the technical or conceptual theme of the slide.

OOP as it Should Be

Level 1

Classes and Objects

Functional (FP) vs Object Oriented (OOP)

University System Example (FP)

- AddStudent
- UpdateStudent
- DeleteStudent
- CalculateAverage
- AddCourse
- UpdateCourse
- DeleteCourse
- EnrollStudentInCourse
- UnEnrollStudentFromCourse
- HowManyStudentsInCourse
- Doctor (Add, Edit, Delete)
- AssignCourse ToDoctor
- SendEmail ToStudent
- Send TextMessage ToStudent
- SendEmailToDoctor
- Send TextMessage ToDoctor
- CallStudent
- CallDoctor
- AddEmployee
- UpdateEmployee
- DeleteEmployee
- CalculateSalary
- PaySalary
-



Simply

You can have thousands of functions in your System!!!!

What is Class? Why we call it Class?

University System Example (OOP)

Class



Class



Class



Class



Class



Class



Class



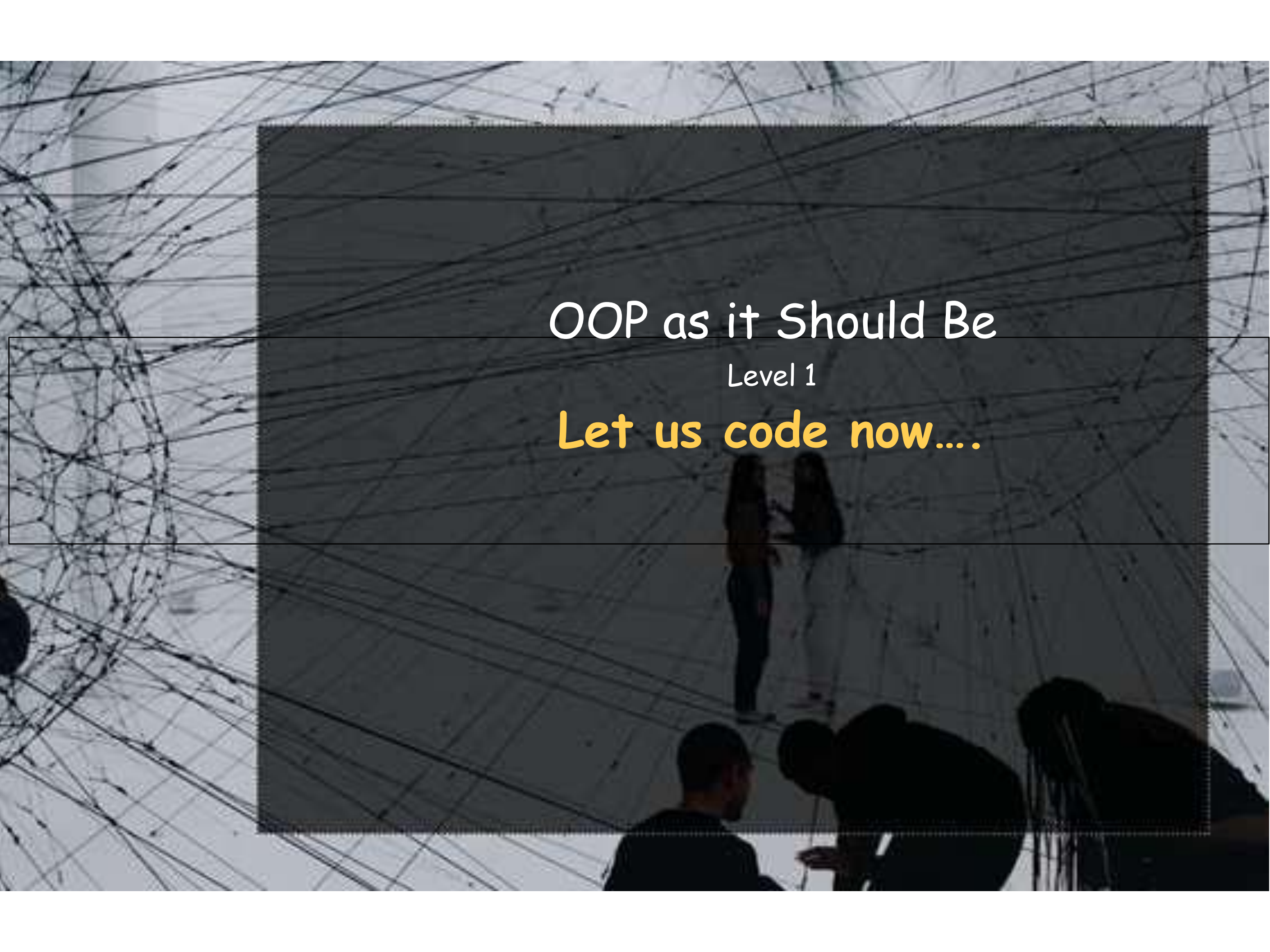
Class



Simply

Class came from
Classification



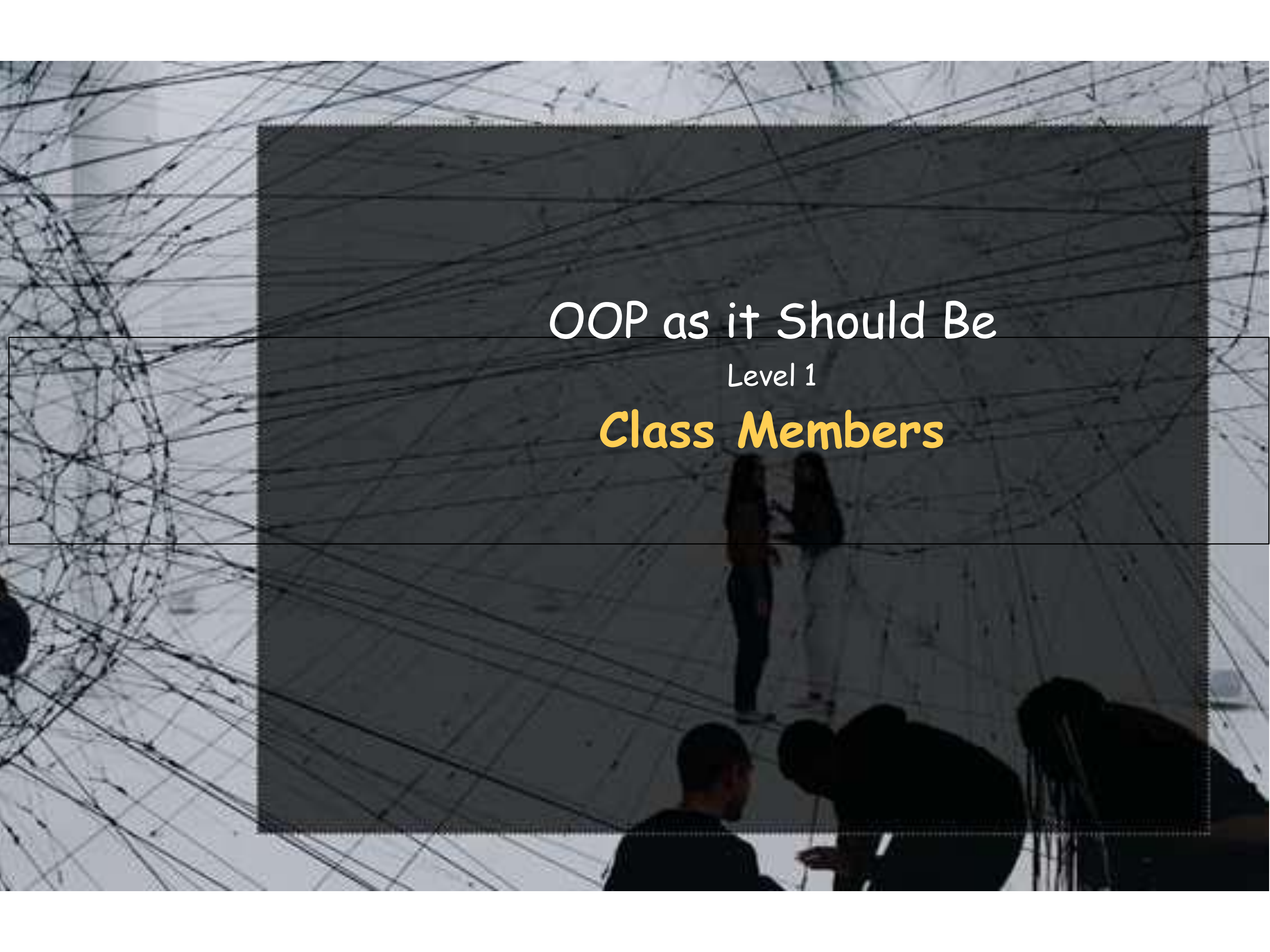
The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning over a table or screen.

OOP as it Should Be

Level 1

Let us code now....

```
1  // Classes and Objects in C++
2  #include <iostream>
3  using namespace std;
4  class clsPerson
5  {
6      // Private by default in classes will not be accessed from outside the class
7      // So we need to make them public
8      public:
9          string FirstName;
10         string LastName;
11         string FullName()
12         {
13             return FirstName + " " + LastName;
14         }
15     };
16     int main()
17     {
18         clsPerson Person1;
19         Person1.FirstName = "Sara";
20         Person1.LastName = "Alterkawi";
21         cout << Person1.FullName() << endl;
22         string S1;
23     }
24
```

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a discussion or collaborative work, adding a human element to the technical theme.

OOP as it Should Be

Level 1

Class Members

Class Members

```
#include <iostream>
using namespace std;

class clsPerson
{
public:

string FirstName;
string LastName;

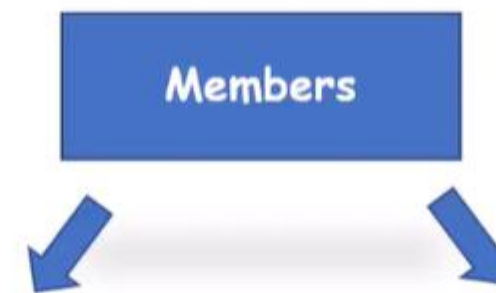
string FullName()
{
    return FirstName + " " + LastName;
}
};

int main()
{
    clsPerson Person1;

    Person1.FirstName = "Mohammed";
    Person1.LastName = "Abu-Hadhoud";

    cout << "Person1: " << Person1.FullName()
    << endl;

}
```



Data Members: Any variable declared inside the class that holds Data.

In Our Case:
FirstName, LastName
Are Data Members

Member Methods(Functions): Any Function Or Procedure declared inside the class.

In Our Case:
FullName()
Is a Member Function/Method

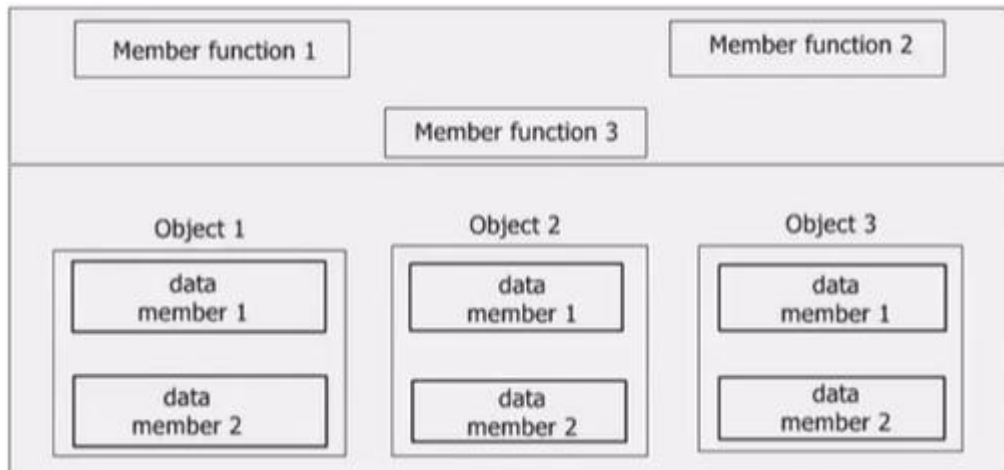
The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in motion, which adds a sense of activity and human presence to the technical theme.

OOP as it Should Be

Level 1

Objects In Memory

Each instance has its own space in memory, Only Member functions are shared among all objects



```
#include <iostream>
using namespace std;

class clsPerson
{
public:
    string FirstName;
    string LastName;

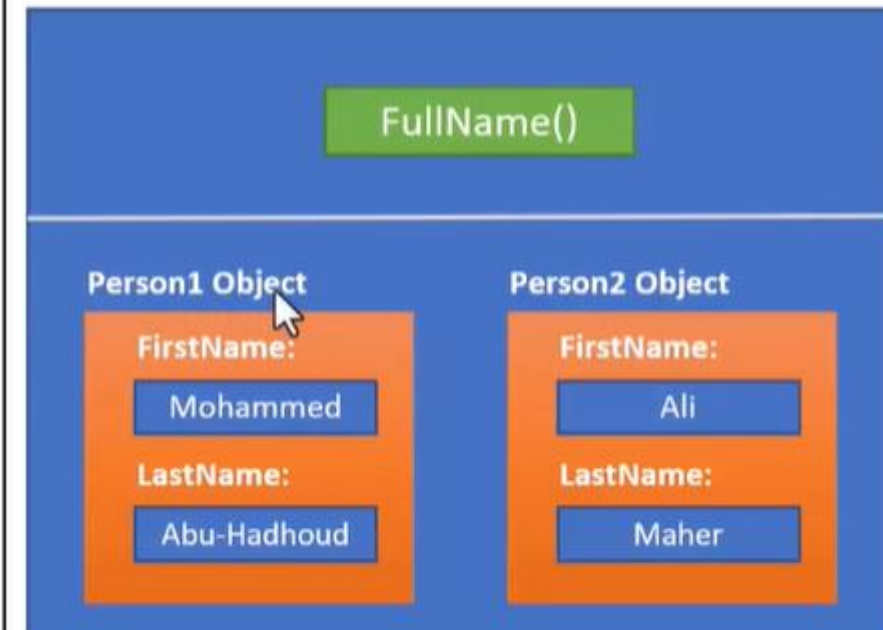
    string FullName()
    {
        return FirstName + " " + LastName;
    }
};

int main()
{
    clsPerson Person1, Person2;

    Person1.FirstName = "Mohammed";
    Person1.LastName = "Abu-Hadhoud";

    Person2.FirstName = "Ali";
    Person2.LastName = "Maher";

    cout << "Person1: " << Person1.FullName() << endl;
    cout << "Person2: " << Person2.FullName() << endl;
}
```



Thank you 😊